ASSIGNMENT

Advanced Java Programming

(Paper 1T1)

First Year M.C.A. — Semester I (CBCS)

RTM Nagpur University

Name: _____

Roll No: _____

Q1. What is Constructor in Java? Explain constructor overloading with suitable program.

A **constructor** is a special method that is called automatically when an object is created using the **new** keyword. It is used to initialize the object's data members.

Features of a Constructor

- Its name is the same as the class name.
- It has no return type, not even void.
- It is called automatically when an object is created.
- If no constructor is written, Java provides a **default constructor**.
- Types: Default (no-argument) constructor and Parameterized constructor.

Constructor Overloading

When a class has **more than one constructor with different parameter lists** (number, type or order of parameters), it is called constructor overloading. Java decides which constructor to call based on the arguments passed while creating the object.

Program

```
class Box {
    double width, height, depth;

    Box() {                                  // default constructor
        width = height = depth = 1;
    }
    Box(double side) {                       // one parameter (cube)
        width = height = depth = side;
    }
    Box(double w, double h, double d) {      // three parameters
        width = w; height = h; depth = d;
    }
    double volume() {
        return width * height * depth;
    }

    public static void main(String[] args) {
        Box b1 = new Box();
        Box b2 = new Box(5);
        Box b3 = new Box(3, 4, 5);
        System.out.println("Volume of b1 = " + b1.volume());
        System.out.println("Volume of b2 = " + b2.volume());
        System.out.println("Volume of b3 = " + b3.volume());
    }
}
```

Output

```
Volume of b1 = 1.0
Volume of b2 = 125.0
Volume of b3 = 60.0
```

Here, Box() is called for b1, Box(double) for b2 and Box(double, double, double) for b3.

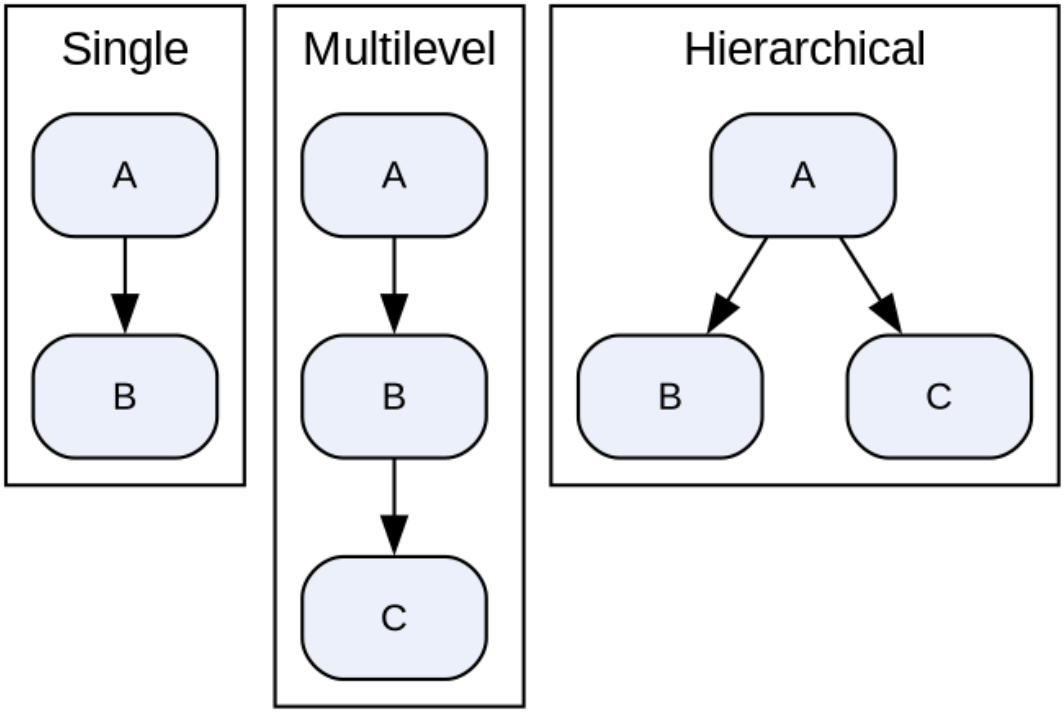
Q2. How many types of inheritance does Java support without interfaces? Explain with sample program.

Inheritance is the mechanism by which one class (subclass) acquires the properties and methods of another class (superclass) using the **extends** keyword. It provides code reusability.

Without interfaces, Java supports **three types** of inheritance using classes:

- **Single inheritance:** One subclass inherits one superclass (A → B).
- **Multilevel inheritance:** A class is derived from a class which is itself derived from another class (A → B → C).
- **Hierarchical inheritance:** Many subclasses inherit the same superclass (A → B, A → C).

Multiple inheritance (one class extending two classes) is **not supported** with classes because of the ambiguity (diamond) problem. It is achieved only through interfaces. Hybrid inheritance also needs interfaces.



Program

```
class Person {                                  // base class
    void showPerson() { System.out.println("I am a Person"); }
}
class Student extends Person {                  // Single: Person -> Student
    void showStudent() { System.out.println("I am a Student"); }
}
class MCAStudent extends Student {              // Multilevel: Person -> Student ->
MCAStudent
    void showMCA() { System.out.println("I am an MCA Student"); }
}
class Teacher extends Person {                  // Hierarchical: Person -> Student,
Teacher
    void showTeacher() { System.out.println("I am a Teacher"); }
}

public class InheritTypes {
```

```
    public static void main(String[] args) {
        System.out.println("--- Single ---");
        Student s = new Student();
        s.showPerson(); s.showStudent();

        System.out.println("--- Multilevel ---");
        MCAStudent m = new MCAStudent();
        m.showPerson(); m.showStudent(); m.showMCA();

        System.out.println("--- Hierarchical ---");
        Teacher t = new Teacher();
        t.showPerson(); t.showTeacher();
    }
}
```

Output

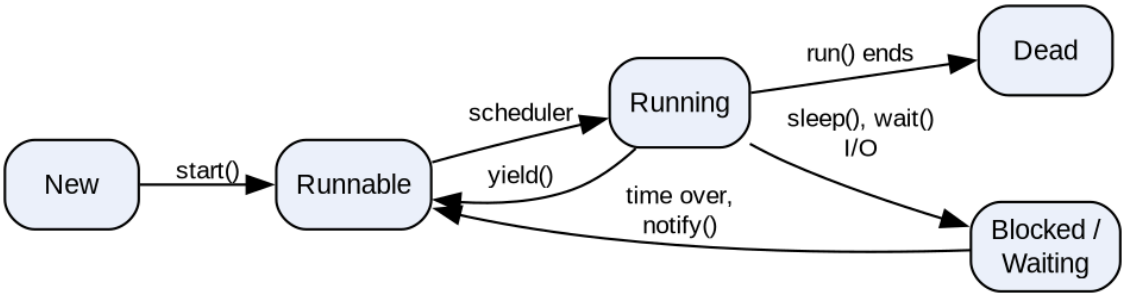
```
--- Single ---
I am a Person
I am a Student
--- Multilevel ---
I am a Person
I am a Student
I am an MCA Student
--- Hierarchical ---
I am a Person
I am a Teacher
```

Q3. What is Multithreading in Java? Explain Life cycle of thread in detail.

Multithreading is a feature of Java that allows two or more parts of a program (threads) to run **concurrently**. A thread is a lightweight sub-process — the smallest unit of execution. Threads share the same memory, so multithreading saves memory and makes better use of the CPU.

- A thread is created by **extending the Thread class** or by **implementing the Runnable interface**.
- Its code is written in the **run()** method and it is started with **start()**.
- Examples: playing music while typing, downloading files while browsing.

Life Cycle of a Thread



- **New (Born):** The thread object is created using new but start() has not been called yet.
- **Runnable (Ready):** After start() is called, the thread is ready to run and waits for the CPU in the thread scheduler's queue.
- **Running:** The scheduler picks the thread and its run() method executes.
- **Blocked / Waiting:** The thread is temporarily inactive because of sleep(), wait(), join() or waiting for I/O or a lock. It goes back to Runnable when the time is over, notify()/notifyAll() is called, or the I/O completes.
- **Dead (Terminated):** The thread finishes run() normally or is stopped by an exception. A dead thread cannot be restarted.

Important Methods

- start(), run(), sleep(ms), join(), isAlive(), setPriority(), getName(), wait(), notify(), notifyAll().

Q4. What is Exception handling? Explain try-catch-finally block with suitable program.

An **exception** is an abnormal condition (run-time error) that disturbs the normal flow of a program, e.g. dividing by zero or accessing an invalid array index. **Exception handling** is the mechanism to handle such run-time errors so that the program does not terminate abnormally.

- All exceptions are subclasses of **Throwable** → Exception / Error.
- **Checked exceptions:** checked at compile time (IOException, SQLException).
- **Unchecked exceptions:** occur at run time (ArithmeticException, NullPointerException).
- Keywords: **try, catch, finally, throw, throws**.

try-catch-finally

- **try:** contains statements that may throw an exception.
- **catch:** handles the exception thrown in the try block. There can be multiple catch blocks.
- **finally:** always executes whether an exception occurs or not; used for clean-up such as closing files or connections.

```
try {
    // code that may throw exception
} catch (ExceptionType e) {
    // handling code
} finally {
    // always executed
}
```

Program

```
public class ExceptionDemo {
    public static void main(String[] args) {
        int a = 10, b = 0;
        try {
            System.out.println("Inside try block");
            int c = a / b;                      // throws ArithmeticException
            System.out.println("Result = " + c);
        }
        catch (ArithmeticException e) {
            System.out.println("Exception caught: " + e.getMessage());
        }
        finally {
            System.out.println("finally block always executes");
        }
        System.out.println("Program continues...");
    }
}
```

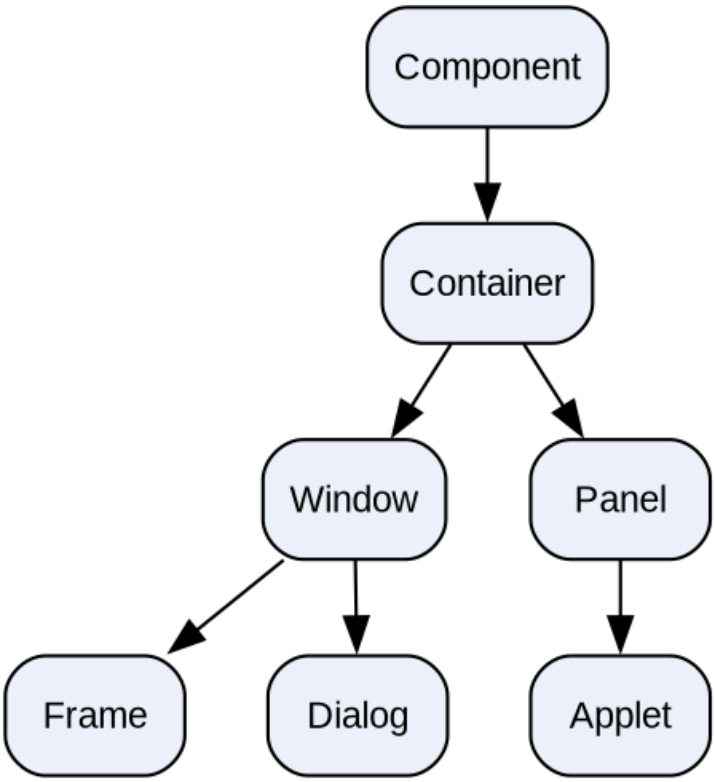
Output

```
Inside try block
Exception caught: / by zero
finally block always executes
Program continues...
```

Q5. Explain the AWT component's working with windows.

AWT (Abstract Window Toolkit) is Java's API (package **java.awt**) to create GUI (window-based) applications. AWT components are platform-dependent (heavyweight) because they use the native OS resources.

AWT Class Hierarchy



- **Component:** Abstract base class of all GUI elements (button, label…). It handles painting and events.
- **Container:** A component that can hold other components, using add().
- **Window:** A top-level window with no title bar and no border.
- **Frame:** A window with a title bar, menu bar, border and resize/close buttons. Most AWT applications use Frame.
- **Dialog:** A pop-up window for messages or input.
- **Panel:** A container with no title bar; used to group components. Applet is a subclass of Panel.

Working with a Frame Window

- Create a frame by **extending Frame** or by creating an object of Frame.
- **setTitle()** sets the title, **setSize(w, h)** the size, **setLayout()** the layout manager.
- **add()** places components such as Label, Button and TextField in the window.
- **setVisible(true)** displays the window.
- A frame does not close by default; a **WindowListener / WindowAdapter** must handle windowClosing().

Program

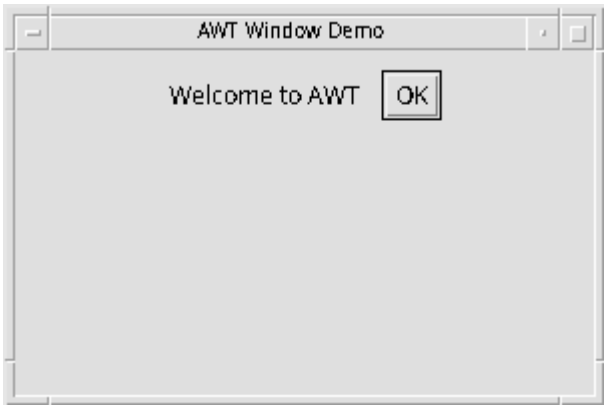
```
import java.awt.*;
import java.awt.event.*;

public class MyFrame extends Frame {
    MyFrame() {
        setTitle("AWT Window Demo");
        setSize(300, 200);
        setLayout(new FlowLayout());
        add(new Label("Welcome to AWT"));
        add(new Button("OK"));

        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();                      // close the window
            }
        });
        setVisible(true);
    }

    public static void main(String[] args) {
        new MyFrame();
    }
}
```

Output



Q6. Explain passing parameter to Applet in detail.

An **applet** can receive values from the HTML page in which it is embedded, just like command-line arguments for an application. This makes the applet flexible — the same applet can show different output without recompiling.

Steps

- Write the **<PARAM>** tag inside the **<APPLET>** tag in the HTML file. Each parameter has a **NAME** and a **VALUE**.
- In the applet, read the value using **getParameter(String name)**. It returns the value as a **String**, or **null** if the parameter is not found.
- Read parameters in the **init()** method.
- Convert to numbers if needed using Integer.parseInt() or Double.parseDouble().

Syntax

```
<applet code="ClassName.class" width="w" height="h">
    <param name="paramName" value="paramValue">
</applet>
```

Program

```
File: Applet.html
<html>
<body>
<applet code="ParamDemo.class" width="400" height="400">
<param name="string" value="Alex">
</applet>
</body>
</html>

File: ParamDemo.java
import java.awt.*;
import java.applet.*;

public class ParamDemo extends Applet
{
    String str;

    public void init()
    {
        str = getParameter("string");
        if (str == null)
            str = "java";
        else
            str = "Hello " + str;
    }

    public void paint(Graphics g)
    {
        g.drawString(str, 10, 100);
    }
}
```

Output (appletviewer Applet.html)

```
Hello Alex
```

If the PARAM tag is removed, getParameter() returns null and the applet displays "java".

Q7. What is Layout Manager? Explain it in detail.

A **Layout Manager** is an object that automatically decides the **position and size** of components inside a container. Every container has a default layout manager, which is changed using **setLayout()**. Layout managers make the GUI independent of screen size and resolution.

Types of Layout Managers

- **FlowLayout:** Arranges components in a row, left to right; moves to the next line when the row is full. Default for **Panel** and **Applet**. Alignment can be LEFT, CENTER or RIGHT.
- **BorderLayout:** Divides the container into five regions — **NORTH, SOUTH, EAST, WEST and CENTER**. Default for **Frame** and **Dialog**.
- **GridLayout:** Arranges components in a grid of equal-sized cells — GridLayout(rows, cols).
- **CardLayout:** Stacks components like a deck of cards; only one card is visible at a time (next(), previous(), show()).
- **GridBagLayout:** The most flexible layout; components can span multiple rows/columns using **GridBagConstraints**.

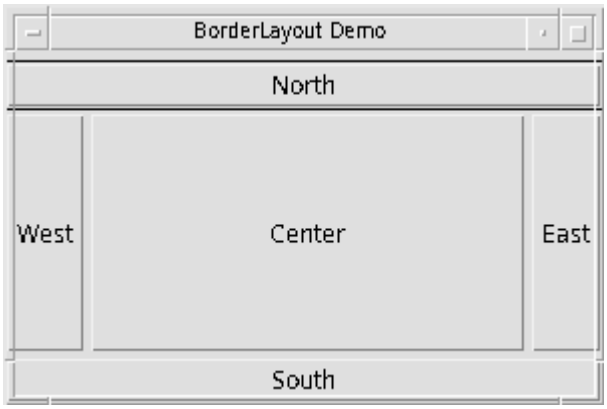
Program (BorderLayout)

```
import java.awt.*;

public class LayoutDemo extends Frame {
    LayoutDemo() {
        setTitle("BorderLayout Demo");
        setLayout(new BorderLayout());
        add(new Button("North"), BorderLayout.NORTH);
        add(new Button("South"), BorderLayout.SOUTH);
        add(new Button("East"), BorderLayout.EAST);
        add(new Button("West"), BorderLayout.WEST);
        add(new Button("Center"), BorderLayout.CENTER);
        setSize(300, 200);
        setVisible(true);
    }

    public static void main(String[] args) {
        new LayoutDemo();
    }
}
```

Output



Q8. Explain and Write a program to demonstrate the addition of a list to scroll pane.

A **JScrollPane** (Swing, package javax.swing) provides a scrollable view of a component. When the content (for example a long list) is larger than the visible area, horizontal and/or vertical **scroll bars** appear automatically.

A **JList** displays a list of items from which the user can select one or more items. A JList does not scroll by itself, so it is added to a JScrollPane.

Steps

- Create an array of items and pass it to the **JList** constructor.
- Create a **JScrollPane** and pass the JList to its constructor: new JScrollPane(list).
- Add the **scroll pane** (not the list) to the frame.
- Scroll bar policy can be set: VERTICAL_SCROLLBAR_ALWAYS, AS_NEEDED or NEVER.

Program

```
import javax.swing.*;

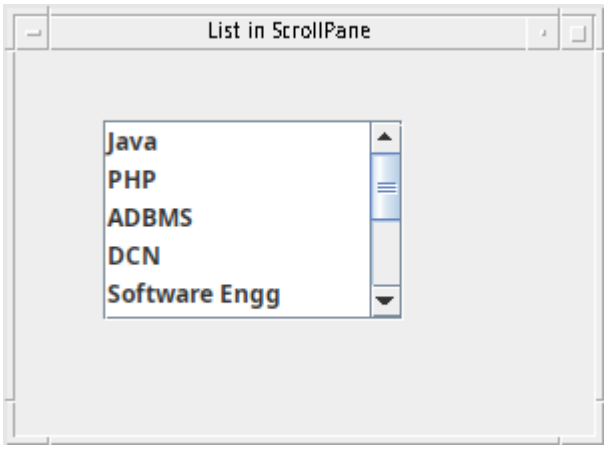
public class ListScrollDemo {
    public static void main(String[] args) {
        JFrame f = new JFrame("List in ScrollPane");

        String[] subjects = { "Java", "PHP", "ADBMS", "DCN", "Software Engg",
                              "C#", "ASP.NET", "Python", "Android", "Cloud" };
        JList<String> list = new JList<>(subjects);

        JScrollPane sp = new JScrollPane(list);    // add list to scroll pane
        sp.setBounds(50, 30, 150, 100);

        f.add(sp);
        f.setSize(300, 220);
        f.setLayout(null);
        f.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        f.setVisible(true);
    }
}
```

Output

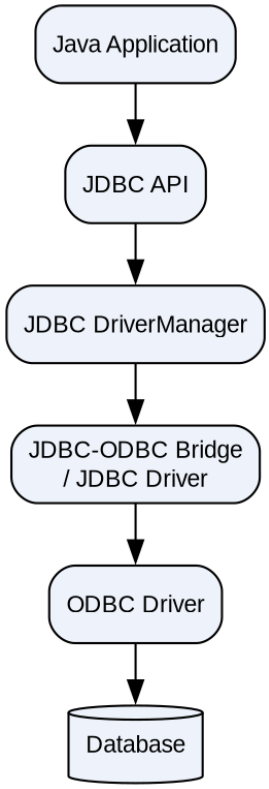


The list shows the first few subjects with a vertical scroll bar; scrolling down shows the remaining subjects.

Q9. Discuss the basic architecture of JDBC-ODBC with various drivers.

JDBC (Java Database Connectivity) is a Java API (package **java.sql**) used to connect a Java program to a database and execute SQL queries. **ODBC** is Microsoft's C-based standard API for databases. The **JDBC-ODBC bridge** converts JDBC calls into ODBC calls.

JDBC Architecture



- **Java Application:** Uses JDBC API calls such as executeQuery().
- **JDBC API:** Interfaces such as Connection, Statement, PreparedStatement, ResultSet.
- **DriverManager:** Loads the correct driver and creates the connection — DriverManager.getConnection(url, user, pwd).
- **JDBC Driver:** Converts JDBC calls into database-specific calls.

Types of JDBC Drivers

Type	Name	Working
Type 1	JDBC-ODBC Bridge driver	Converts JDBC calls into ODBC calls; needs ODBC driver (DSN) on the client. Slow. Removed from Java 8.
Type 2	Native-API driver	Converts JDBC calls into the database's native client library calls (e.g. Oracle OCI).
Type 3	Network Protocol driver	Sends calls to a middleware server, which converts them for the database.
Type 4	Thin (Pure Java) driver	Converts JDBC calls directly into the database protocol. Fastest and most used (MySQL Connector/J, Oracle Thin).

Steps to Connect

- Load driver: Class.forName("sun.jdbc.odbc.JdbcOdbcDriver") (Type 1)
- Connect: DriverManager.getConnection("jdbc:odbc:mydsn")
- Create Statement → execute query → process ResultSet → close connection.

Q10. Why are domain names required? What is the relationship between domain names and IP address.

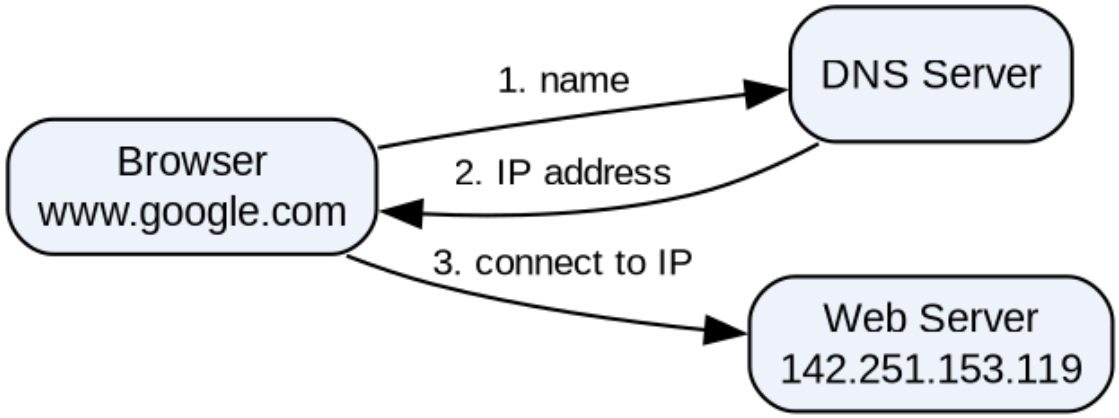
Every computer on the Internet is identified by a numeric **IP address** (e.g. 142.250.183.4). IP addresses are hard for people to remember, so a human-friendly name called a **domain name** (e.g. www.google.com) is used.

Why Domain Names are Required

- **Easy to remember:** names like www.rtmnu.ac.in are easier than numbers.
- **Stable identity:** a website's IP address may change (new server), but its domain name stays the same.
- **One name, many servers:** one domain can map to several IP addresses for load balancing.
- **Meaningful structure:** shows organisation and type — .com, .edu, .ac.in, .gov.

Relationship between Domain Name and IP Address

- A domain name is a **logical name**; the IP address is the **actual network address**.
- The **DNS (Domain Name System)** is a distributed database that **translates domain names into IP addresses** (name resolution). Reverse lookup converts an IP back to a name.
- When a user types a URL, the browser asks the DNS server for the IP, then connects to that IP.



Java Example (InetAddress class)

```
import java.net.*;

public class DNSDemo {
    public static void main(String[] args) throws Exception {
        InetAddress local = InetAddress.getLocalHost();
        System.out.println("Local host : " + local);

        InetAddress addr = InetAddress.getByName("www.google.com");
        System.out.println("Host name  : " + addr.getHostName());
        System.out.println("IP address : " + addr.getHostAddress());
    }
}
```

Output (IP address may differ)

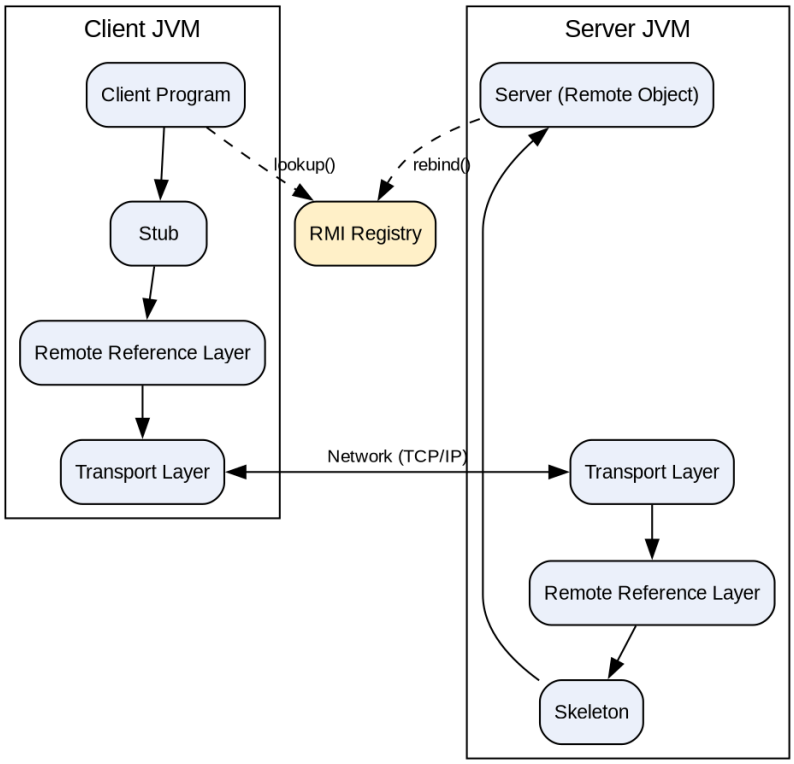
```
Local host : asus/127.0.1.1
Host name  : www.google.com
```

```
IP address : 142.251.153.119
```

Q11. Describe RMI architecture in detail.

RMI (Remote Method Invocation) is a Java API (package **java.rmi**) that allows an object running in one JVM to call methods of an object running in **another JVM** (on the same or a different machine). It is used to build distributed applications.

RMI Architecture



- **Client & Server Programs:** The server creates the remote object and registers it; the client looks it up and calls its methods.
- **Stub (client side):** A proxy for the remote object. It **marshals** (packs) the method name and parameters and sends them to the server, then **unmarshals** the result.
- **Skeleton (server side):** Receives the request, unmarshals the parameters, calls the actual method on the remote object, and sends back the result. (From Java 2 onwards the skeleton is not needed.)
- **Remote Reference Layer (RRL):** Manages references to remote objects and the invocation semantics.
- **Transport Layer:** Sets up the TCP/IP connection between client and server and transfers the data.
- **RMI Registry:** A naming service where the server binds the object with a name — **Naming.rebind("name", obj)** — and the client finds it with **Naming.lookup("name")**.

Steps to Create an RMI Application

- Create a remote interface that extends **Remote**; each method throws **RemoteException**.
- Implement the interface in a class that extends **UnicastRemoteObject**.
- Compile the classes (use rmic for stub/skeleton in older JDKs).
- Start **rmiregistry**, run the server, then run the client.

Q12. Write a short note on: 1) TCP/IP client socket 2) TCP/IP server socket

A **socket** is one end-point of a two-way communication link between two programs on a network. A socket is identified by an **IP address + port number**. TCP/IP sockets are **connection-oriented and reliable**: data arrives in order without loss. Java provides them in the **java.net** package.

1) TCP/IP Client Socket (Socket class)

The **Socket** class is used by the **client** to connect to a server. Creating a Socket object automatically establishes the connection.

- **Socket(String host, int port)** — connects to the given host name and port.
- **Socket(InetAddress ip, int port)** — connects using an InetAddress object.
- **getInputStream()** / **getOutputStream()** — streams to read and write data.
- **getInetAddress(), getPort(), getLocalPort()** — connection details.
- **close()** — closes the socket.

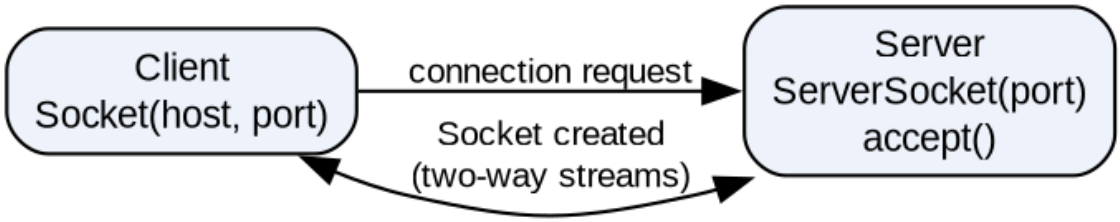
```
Socket s = new Socket("localhost", 420);
OutputStream out = s.getOutputStream();
out.write("good morning mca".getBytes());
s.close();
```

2) TCP/IP Server Socket (ServerSocket class)

The **ServerSocket** class is used by the **server** to listen for client connections on a port.

- **ServerSocket(int port)** — creates a server socket on the port.
- **ServerSocket(int port, int maxQueue)** — also sets the maximum number of waiting clients.
- **accept()** — waits (blocks) until a client connects and returns a **Socket** for that client.
- **close()** — closes the server socket.

```
ServerSocket ss = new ServerSocket(420);
Socket s = ss.accept();          // wait for client
InputStream in = s.getInputStream();
```



Q13. What is Networking? Explain Datagram in detail.

Networking means connecting two or more computing devices so that they can share data and resources. Java networking (package **java.net**) lets programs communicate over the Internet using **TCP** (connection-oriented) or **UDP** (connectionless). Important classes: InetAddress, URL, URLConnection, Socket, ServerSocket, DatagramSocket and DatagramPacket.

Datagram

A **datagram** is an independent, self-contained message sent over the network using **UDP (User Datagram Protocol)**. Its arrival, arrival time and order are **not guaranteed**. Datagrams are fast and are used for video streaming, online games, DNS, etc.

DatagramPacket

A container for the data, with the destination address and port.

- **DatagramPacket(byte[] buf, int len)** — for receiving data.
- **DatagramPacket(byte[] buf, int len, InetAddress ip, int port)** — for sending data.
- Methods: getData(), getLength(), getAddress(), getPort().

DatagramSocket

A socket used to send and receive datagram packets.

- **DatagramSocket()** / **DatagramSocket(int port)**
- **send(DatagramPacket p)**, **receive(DatagramPacket p)**, **close()**

TCP vs UDP (Datagram)

TCP	UDP (Datagram)
Connection-oriented	Connectionless
Reliable, ordered delivery	No guarantee of delivery or order
Slower	Faster
Socket, ServerSocket	DatagramSocket, DatagramPacket

Example

```
// Sender
DatagramSocket ds = new DatagramSocket();
byte[] b = "good morning".getBytes();
DatagramPacket dp = new DatagramPacket(b, b.length,
                    InetAddress.getByName("localhost"), 3000);
ds.send(dp);

// Receiver
DatagramSocket ds = new DatagramSocket(3000);
byte[] b = new byte[100];
DatagramPacket dp = new DatagramPacket(b, b.length);
ds.receive(dp);
System.out.println(new String(dp.getData(), 0, dp.getLength()));
```

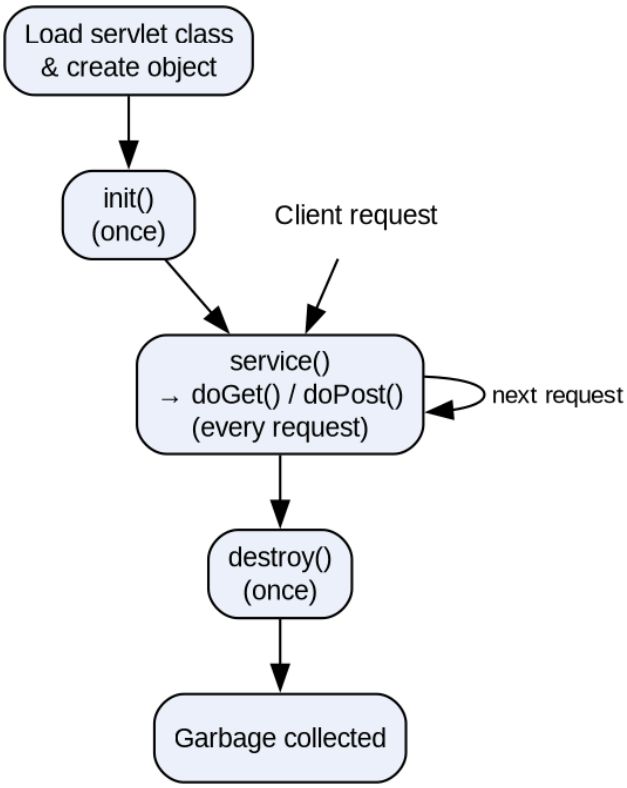
Q14. What is Servlet? Explain Life cycle of servlet with block diagram.

A **Servlet** is a Java program that runs on a **web server** (servlet container such as **Apache Tomcat**). It receives HTTP requests from the client (browser), processes them, and generates a dynamic response (usually HTML). Servlets are defined in the **javax.servlet** and **javax.servlet.http** packages.

- Platform independent, faster than CGI (one instance, multiple threads), secure and robust.

Life Cycle of a Servlet

The life cycle is managed by the servlet container and has these stages:



- **Loading and Instantiation:** The container loads the servlet class and creates **one** object of it (on the first request or at server start-up).
- **init():** Called **only once** after the object is created. Used for initialization, such as opening a database connection.
- **service():** Called for **every request**. In HttpServlet it calls **doGet()** or **doPost()** depending on the request type. Each request runs in a separate thread.
- **destroy():** Called **only once** before the servlet is removed (server shutdown/undeploy). Used to release resources.
- After destroy(), the object is garbage collected.

Example

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LifeServlet extends HttpServlet {
    public void init() throws ServletException {
        System.out.println("init() called - servlet loaded");
```

```
    }
    public void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        res.getWriter().println("<h2>Hello from Servlet</h2>");
    }
    public void destroy() {
        System.out.println("destroy() called - servlet removed");
    }
}
```

Q15. What is Introspection? Explain developing beans using BDK.

Introspection

Introspection is the process by which a builder tool (like the BDK BeanBox) **automatically analyzes a Java Bean** to discover its **properties, events and methods** at design time. This allows the bean to be used visually without reading its source code. Introspection is done by the **Introspector** class (package **java.beans**) in two ways:

- **Design patterns (naming conventions):** Methods like getXxx()/setXxx() identify property xxx; isXxx() for boolean properties; addXxxListener()/removeXxxListener() identify events.
- **BeanInfo interface:** The developer writes a class named **<BeanName>BeanInfo** that explicitly describes properties (PropertyDescriptor), events (EventSetDescriptor) and methods (MethodDescriptor).

BDK (Bean Development Kit)

The **BDK** is a tool from Sun Microsystems for creating, configuring and testing Java Beans. Its **BeanBox** window is used to place beans, change their properties and connect events.

Steps to Develop a Bean using BDK

- **1. Create a directory** for the bean, e.g. C:\bdk\demo\sunw\demo\colors.
- **2. Write the Java source** — a public class with a no-argument constructor, private properties and public get/set methods; it implements Serializable.
- **3. Compile** the source file using javac.
- **4. Create a manifest file** (colors.mft) that names the bean class: Name: sunw/demo/colors/Colors.class and Java-Bean: True.
- **5. Generate a JAR file**: jar cfm ..\jars\colors.jar colors.mft sunw\demo\colors*.class
- **6. Start the BDK** (run.bat) — the bean appears in the **ToolBox**.
- **7. Test the bean**: drag it into the **BeanBox**, change its properties in the **Properties** window and connect events.

Simple Bean

```
public class Student implements java.io.Serializable {
    private String name;
    public Student() { }                       // no-arg constructor
    public String getName() { return name; }   // getter
    public void setName(String n) { name = n; } // setter
}
```

Q16. Explain cookies and session tracking with suitable example.

HTTP is a **stateless** protocol — the server does not remember the client between requests. **Session tracking** is a technique to maintain the state (data) of a user across multiple requests. Techniques: **Cookies, HttpSession, URL Rewriting and Hidden Form Fields**.

Cookies

A **cookie** is a small piece of information (name–value pair) that the server sends to the browser. The browser **stores it on the client** and sends it back with every request to the same server.

- Create: Cookie ck = new Cookie("user", "Rahul");
- Send to client: response.addCookie(ck);
- Read: Cookie[] c = request.getCookies();
- Expiry: ck.setMaxAge(seconds); setMaxAge(0) deletes the cookie.
- Types: **Non-persistent** (deleted when the browser closes) and **Persistent** (stored until the expiry time).
- Disadvantage: does not work if cookies are disabled; only text data can be stored.

Session Tracking using HttpSession

The **HttpSession** object is **stored on the server**. Each user gets a unique **session ID** (sent to the browser as the JSESSIONID cookie).

- HttpSession session = request.getSession();
- session.setAttribute("name", value); / session.getAttribute("name");
- session.getId(), session.setMaxInactiveInterval(sec), session.invalidate() (logout).

Example

```
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class CookieServlet extends HttpServlet {
    public void doGet(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        res.setContentType("text/html");
        PrintWriter out = res.getWriter();

        // Cookie: stored on the client (browser)
        Cookie ck = new Cookie("user", "Rahul");
        ck.setMaxAge(3600);                       // 1 hour
        res.addCookie(ck);

        // Session: stored on the server
        HttpSession session = req.getSession();
        Integer count = (Integer) session.getAttribute("count");
        count = (count == null) ? 1 : count + 1;
        session.setAttribute("count", count);

        out.println("<h3>Cookie 'user' added</h3>");
        out.println("Session ID : " + session.getId() + "<br>");
        out.println("You visited this page " + count + " time(s)");
    }
}
```

Output (on the second visit)

```
Cookie 'user' added
Session ID : 7F3A2C9B1D...
You visited this page 2 time(s)
```

Q17. What is the difference between Javabeans and EJB?

JavaBeans are reusable software components for building applications, mainly used on the client side or in the presentation layer. **EJB (Enterprise JavaBeans)** are server-side components that run inside an **EJB container** of an application server and contain the business logic of enterprise applications.

JavaBeans	**Enterprise JavaBeans (EJB)**
Reusable components that can be visually manipulated in a builder tool.	Server-side components that contain business logic.
Run in any JVM; no container needed.	Run inside an EJB container (JBoss, WebLogic, GlassFish).
Mostly used on the client side (GUI, JSP data).	Used on the server side in distributed enterprise applications.
Package: java.beans.	Package: javax.ejb.
Not distributed — cannot be accessed remotely by default.	Distributed — can be accessed remotely (RMI/IIOP).
No built-in services.	Container provides transactions, security, persistence, pooling.
Types: visible beans (GUI) and invisible beans.	Types: Session Bean, Entity Bean, Message-Driven Bean.
Simple: class with no-arg constructor, getters/setters, Serializable.	More complex: needs interfaces, annotations/deployment descriptor.
Example: a Button bean, Student bean used in JSP.	Example: bank transaction or online shopping-cart bean.

Conclusion: JavaBeans are simple reusable components, while EJBs are heavyweight server-side components for large, secure, transactional enterprise applications.

Q18. Explain types of JSP tags? Write a program in Java which demonstrate use of JSP and Servlet.

JSP (Java Server Pages) is a server-side technology used to create dynamic web pages by embedding Java code in HTML. A JSP page is translated into a servlet by the container.

Types of JSP Tags

Tag	Syntax	Use
Directive	<%@ page import="java.util.*" %>	Gives instructions to the container: page, include, taglib.
Declaration	<%! int count = 0; %>	Declares variables and methods (class level).
Scriptlet	<% count++; %>	Writes Java statements inside the service method.
Expression	<%= new java.util.Date() %>	Evaluates an expression and prints it to the output.
Comment	<%-- comment --%>	JSP comment; not sent to the client.
Action	<jsp:include>, <jsp:forward>, <jsp:useBean>	Built-in actions for including pages, forwarding and beans.

Program: JSP + Servlet (Login)

The HTML form sends the name to a **Servlet**; the servlet forwards it to a **JSP** page, which displays it.

```
File: index.html
<form action="LoginServlet" method="post">
    Name: <input type="text" name="uname">
    <input type="submit" value="Login">
</form>

File: LoginServlet.java
import java.io.*;
import javax.servlet.*;
import javax.servlet.http.*;

public class LoginServlet extends HttpServlet {
    public void doPost(HttpServletRequest req, HttpServletResponse res)
            throws ServletException, IOException {
        String name = req.getParameter("uname");
        req.setAttribute("name", name);
        RequestDispatcher rd = req.getRequestDispatcher("welcome.jsp");
        rd.forward(req, res);                    // pass control to JSP
    }
}

File: welcome.jsp
<%@ page contentType="text/html" %>
<html>
<body>
    <h2>Welcome, <%= request.getAttribute("name") %></h2>
    <% java.util.Date d = new java.util.Date(); %>
    Login time: <%= d %>
</body>
</html>
```

Output

```
Welcome, Rahul
Login time: Sat Oct 31 21:30:00 IST 2026
```

Q19. Differentiate between doGet() and doPost() methods.

doGet() and **doPost()** are methods of the **HttpServlet** class. They handle HTTP **GET** and **POST** requests respectively. Both have the signature:

```
protected void doGet(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException
protected void doPost(HttpServletRequest req, HttpServletResponse res)
        throws ServletException, IOException
```

doGet()	**doPost()**
Handles HTTP GET requests.	Handles HTTP POST requests.
Data is sent in the URL as a query string (?name=Rahul&age=22).	Data is sent in the body of the request.
Data is visible in the address bar — less secure.	Data is not visible in the URL — more secure.
Limited data size (URL length limit, about 2048 characters).	No size limit; large data and files can be sent.
Only text (ASCII) data.	Text and binary data (file upload).
Can be bookmarked and cached.	Cannot be bookmarked or cached.
Idempotent — used to fetch data (search, view).	Non-idempotent — used to send/update data (login, registration).
Default method of HTML form and links.	Must be set with method="post" in the form.
Faster.	Slightly slower.

Example

```
<form action="MyServlet" method="get">   → calls doGet()
<form action="MyServlet" method="post">  → calls doPost()
```

Q20. Explain in detail: 1) JAR File 2) Advantages of Java beans

1) JAR File

A **JAR (Java ARchive)** file is a compressed file (based on ZIP format) that **packages many Java class files, images, sounds and other resources into a single file** with the .jar extension. It is used to distribute Java applications, libraries, applets and **Java Beans**.

- **Compression:** reduces size, so download is faster.
- **Single file:** a whole application is distributed as one file.
- **Manifest file (META-INF/MANIFEST.MF):** describes the contents — e.g. Main-Class for executable JARs, and Java-Bean: True for beans.
- **Security:** JAR files can be digitally signed.
- **Executable JAR:** can be run with java -jar app.jar.

JAR Commands

Command	Purpose
jar cf myapp.jar *.class	Create a JAR file
jar cfm myapp.jar manifest.mft *.class	Create a JAR with a manifest
jar tf myapp.jar	List the contents
jar xf myapp.jar	Extract the files
java -jar myapp.jar	Run an executable JAR

2) Advantages of Java Beans

A **Java Bean** is a reusable software component written in Java that can be manipulated visually in a builder tool.

- **Reusability:** "Write once, run anywhere, reuse everywhere" — the same bean can be used in many applications.
- **Platform independent:** beans run on any platform that has a JVM.
- **Visual development:** properties, events and methods can be configured in builder tools (BDK BeanBox) without writing code.
- **Introspection:** builder tools discover a bean's features automatically.
- **Customization:** a bean's appearance and behaviour can be changed through property editors and customizers.
- **Persistence:** beans implement Serializable, so their state can be saved and restored.
- **Event handling:** beans can send and receive events, so they connect easily with other beans.
- **Easy distribution:** beans are packaged in JAR files.